PYTHON · PANDAS

Pandas Practice Workbook

Two End-to-End Data Analytics Projects

Project 1 · Inventory Management Analytics

Project 2 · HR Employee Analytics System

Each project contains a problem statement, sample datasets, a set of functions to implement, interview questions, and challenge extensions.

(Questions only — no solutions included.)

How to Use This Workbook

This document describes what each function must accomplish. Your job is to write the Python (Pandas) code yourself. For every task you will find:

- The function signature you should implement.
- A plain-English explanation of what the function is expected to do.
- A step-by-step breakdown of the required processing.
- A sample input and the exact output your function should produce.

> **Reading the examples**
> Output tables show only the most relevant columns so the expected result is easy to verify. Your returned DataFrame may legitimately contain additional columns from the source data unless a task says otherwise.

Project 1 — Inventory Management Analytics

Problem Statement

A retail company manages inventory across several product categories such as Electronics, Furniture, and Stationery. The inventory team records stock levels, product prices, and supplier information in CSV files.

In practice, the raw data is messy. It typically suffers from the following problems:

- Duplicate product records (the same product entered more than once).
- Missing stock values (blank cells in the Stock column).
- Supplier information stored in a separate file, not joined to the products.
- No easy way to spot products that are running low on stock.
- No reporting on the monetary value tied up in inventory.

Management wants a Python-based analytics system that can clean the data, merge supplier information, flag inventory risks, and produce category-wise reports.

Objective

Design and implement a set of Pandas functions that:

- Load inventory data from a CSV file.
- Clean the inventory records.
- Identify low-stock products.
- Calculate the value of inventory on hand.
- Generate category-wise summary reports.
- Merge supplier information into the inventory data.
- Find the top products by inventory value.

Sample Data

FILE: INVENTORY.CSV

ProductID	ProductName	Category	Stock	Price	SupplierID
P101	Laptop	Electronics	25	55000	S1
P102	Mouse	Electronics	100	500	S2
P103	Keyboard	Electronics	(blank)	1200	S2
P104	Chair	Furniture	15	4500	S3
P104	Chair	Furniture	15	4500	S3

ProductID	ProductName	Category	Stock	Price	SupplierID
P105	Table	Furniture	8	7000	S3
P106	Notebook	Stationery	200	50	S4

Notice the deliberate data issues
Row for `P103 (Keyboard)` has a missing Stock value, and `P104 (Chair)` appears twice. Your cleaning function must handle both of these.

FILE: SUPPLIERS.CSV

SupplierID	SupplierName	City
S1	TechWorld	Mumbai
S2	GadgetHub	Pune
S3	FurnitureMart	Nashik
S4	OfficeStore	Pune

The `SupplierID` column is the link between the two files — it appears in both and lets you match each product to its supplier.

Task 1 · Load and Clean the Inventory Data

FUNCTION

```
def load_and_clean_inventory(inventory_path):
```

PARAMETERS

Parameter	Type	Description
inventory_path	string	Path to the inventory CSV file.

WHAT YOU NEED TO DO

Read the inventory file into a DataFrame and clean it so the data is reliable for the later tasks. “Cleaning” here means removing duplicates, dealing with the missing stock value, and making sure Stock is stored as a whole number.

STEP BY STEP

1. Read the CSV file at `inventory_path` into a DataFrame.
2. Remove duplicate rows so each product appears only once (the repeated Chair row should collapse to one).
3. Replace any missing Stock value with `0`.
4. Convert the Stock column to an integer type (it should not be a decimal).

RETURN

The cleaned DataFrame.

EXPECTED OUTPUT (KEY COLUMNS)

ProductID	ProductName	Stock
P101	Laptop	25
P102	Mouse	100
P103	Keyboard	0
P104	Chair	15
P105	Table	8
P106	Notebook	200

> **Why this matters**
> Keyboard’s blank stock became 0 and the duplicate Chair row is gone. Every later task in this project assumes the data has already been cleaned by this function.

Task 2 · Identify Low-Stock Products

FUNCTION

```
def low_stock_products(inventory_df, threshold):
```

PARAMETERS

Parameter	Type	Description
inventory_df	DataFrame	The cleaned inventory data from Task 1.
threshold	integer	The stock level used as the cut-off.

WHAT YOU NEED TO DO

Return only the products that are running low — that is, products whose Stock is strictly below the given threshold. This helps the team decide what to reorder.

EXAMPLE

Input: `low_stock_products(inventory_df, 20)`

Output (products with stock below 20):

ProductName	Stock
Chair	15
Table	8

> **Read the boundary carefully**
> "Below threshold" means less than the value, not less-than-or-equal. With a threshold of 20, a product with exactly 20 in stock would NOT be flagged.

Task 3 · Calculate Inventory Value

FUNCTION

```
def calculate_inventory_value(inventory_df):
```

WHAT YOU NEED TO DO

Add a new column called `InventoryValue` that shows how much money is tied up in each product's stock. The value for each row is its stock quantity multiplied by its unit price.

FORMULA

```
InventoryValue = Stock × Price
```

EXAMPLE

Product	Stock	Price	InventoryValue
Laptop	25	55000	1375000

How to read the example

For the Laptop: 25 units × ₹55,000 = ₹1,375,000. Your function should compute this for every product and return the DataFrame with the new column added.

Task 4 · Category-wise Summary Report

FUNCTION

```
def category_summary(inventory_df):
```

WHAT YOU NEED TO DO

Group the products by their Category and produce one summary row per category. For each category, calculate three figures:

- Total Stock — the sum of all stock quantities in that category.
- Average Price — the mean unit price of products in that category.
- Total Inventory Value — the combined inventory value of all products in that category.

EXPECTED OUTPUT

Category	TotalStock	AveragePrice	TotalInventoryValue
Electronics	125	18900	1425000
Furniture	23	5750	123000
Stationery	200	50	10000

> **Tip**
> This is a grouping-and-aggregation task: one input category produces one output row. The result is a small summary table, not the full product list.

Task 5 · Merge Supplier Details

FUNCTION

```
def merge_supplier_details(inventory_df, supplier_df):
```

WHAT YOU NEED TO DO

Combine the inventory data with the supplier data so that each product row also shows its supplier's name and city. The two tables share the `SupplierID` column, which is the key used to match them.

EXPECTED OUTPUT (KEY COLUMNS)

ProductName	SupplierName	City
Laptop	TechWorld	Mumbai
Mouse	GadgetHub	Pune

What a merge does

A merge lines up rows from two tables based on a shared key. Here, every product's `SupplierID` is looked up in the supplier table to pull in the matching SupplierName and City.

Task 6 · Find Top Inventory Products

FUNCTION

```
def top_inventory_products(inventory_df, n):
```

PARAMETERS

Parameter	Type	Description
inventory_df	DataFrame	Inventory data that includes an InventoryValue column.
n	integer	How many top products to return.

WHAT YOU NEED TO DO

Return the `n` products that have the highest inventory value, sorted from highest to lowest.

EXAMPLE

Input: `top_inventory_products(inventory_df, 3)`

Output — the 3 products with the largest InventoryValue, in descending order:

Product	InventoryValue
Laptop	1375000
Table	56000
Mouse	50000

> **Make sure the order is correct**
> The result must be sorted by InventoryValue from largest to smallest. The top value (Laptop) should appear first.

Interview Questions

Be ready to explain the reasoning behind common Pandas operations used in this project.

Q1. Why use drop_duplicates()?

Answer: To remove repeated product records so each product is counted only once.

Q2. Why use fillna(0)?

Answer: To replace missing stock values with zero so calculations don't break or produce blanks.

Q3. Why use groupby()?

Answer: To generate category-wise summaries by collapsing many product rows into one row per category.

Q4. Why use merge()?

Answer: To combine the inventory and supplier data into a single table using a shared key.

Q5. Why use nlargest()?

Answer: To pull out the top-N products by value quickly, without sorting the whole dataset by hand.

Challenge Extensions

Once the core tasks work, try these harder variations.

LEVEL 2

```
def supplier_summary(merged_df):
```

Using the merged data from Task 5, produce a supplier-wise report: for each supplier, show the total inventory value of all the products they supply.

LEVEL 3

```
def category_pivot(inventory_df):
```

Generate a category-wise pivot report — a reshaped summary table organised by category. (A pivot table rearranges data so categories become rows and chosen metrics become columns.)

LEVEL 4

```
def stock_ranking(inventory_df):
```

Add a new column `StockRank` that ranks every product by its stock level. (Ranking assigns positions: 1st, 2nd, 3rd, and so on.)

Project 2 — HR Employee Analytics System

Problem Statement

A growing organization maintains employee records across multiple departments such as IT, HR, Finance, and Marketing. The HR team stores employee information in CSV files containing employee details, salary information, experience levels, department assignments, and office locations.

As with most real HR data, the dataset is imperfect. It contains:

- Missing salaries (blank salary cells).
- Missing experience values.
- Duplicate employee records.
- Inconsistent reporting requirements.

Management wants an analytics system that can clean the employee data and produce reports such as department-wise salary analysis, lists of experienced employees, salary-band classification, top-paid employees, and location-wise workforce reports.

Objective

Design and implement a set of Pandas functions that:

- Load and clean the employee data.
- Generate department-wise salary reports.
- Filter employees by experience.
- Classify employees into salary bands.
- Find the top-paid employees.
- Generate location-wise workforce summaries.
- Rank employees by salary.

Sample Data

FILE: EMPLOYEES.CSV

EmpID	Name	Department	Salary	Experience	Location
E101	Ravi	IT	65000	3	Mumbai
E102	Meera	HR	55000	5	Pune
E103	Karan	IT	(blank)	2	Mumbai
E104	Sneha	Finance	75000	7	Nashik
E105	Amit	IT	62000	4	Pune

EmpID	Name	Department	Salary	Experience	Location
E105	Amit	IT	62000	4	Pune
E106	Priya	HR	58000	(blank)	Mumbai
E107	Rohan	Marketing	48000	1	Pune

Deliberate data issues

Karan (E103) is missing a Salary, Priya (E106) is missing an Experience value, and Amit (E105) is duplicated. Your cleaning function must resolve all three.

Task 1 · Load and Clean the Employee Data

FUNCTION

```
def clean_employee_data(employee_path):
```

PARAMETERS

Parameter	Type	Description
employee_path	string	Path to the employees CSV file.

WHAT YOU NEED TO DO

Read the employee file and clean it. Notice that the two missing columns are handled differently: missing salaries are filled with the average salary, while missing experience is filled with zero.

STEP BY STEP

1. Read the CSV file at `employee_path`.
2. Remove duplicate rows (the repeated Amit row should collapse to one).
3. Replace any missing Salary with the mean (average) salary of the dataset.
4. Replace any missing Experience with `0`.
5. Make sure Salary and Experience are stored as numeric types.

RETURN

The cleaned DataFrame.

EXPECTED OUTPUT (THE TWO REPAIRED ROWS)

EmpID	Name	Salary	Experience
E103	Karan	60500	2
E106	Priya	58000	0

> **Where 60500 comes from**
> Karan's missing salary is filled with the average of the other employees' salaries, which works out to 60,500. Priya's missing experience simply becomes 0.

Task 2 · Department Salary Report

FUNCTION

```
def department_salary_report(employee_df):
```

WHAT YOU NEED TO DO

Group employees by their Department and produce one summary row per department. For each department, calculate four figures:

- Average Salary.
- Maximum Salary.
- Minimum Salary.
- Employee Count (how many employees are in that department).

EXPECTED OUTPUT

Department	AvgSalary	MaxSalary	MinSalary	EmployeeCount
IT	62500	65000	60500	3
HR	56500	58000	55000	2

> **Tip**
> This is a grouping task with several aggregations at once. Each department becomes a single row in the report.

Task 3 · Filter Experienced Employees

FUNCTION

```
def experienced_employees(employee_df, min_exp):
```

PARAMETERS

Parameter	Type	Description
employee_df	DataFrame	The cleaned employee data.
min_exp	integer	The minimum years of experience required.

WHAT YOU NEED TO DO

Return the employees whose experience is greater than or equal to `min_exp` years.

EXAMPLE

Input: `experienced_employees(employee_df, 5)`

Output (employees with 5 or more years of experience):

Name	Experience
Meera	5
Sneha	7

> **Watch the boundary**
> Unlike the low-stock task, this one is greater-than-OR-equal. An employee with exactly 5 years (Meera) IS included.

Task 4 · Salary Band Classification

FUNCTION

```
def salary_band(employee_df):
```

WHAT YOU NEED TO DO

Add a new column called `SalaryBand` that labels each employee as High, Medium, or Low based on their salary, using the rules below.

CLASSIFICATION RULES

Condition	SalaryBand
Salary ≥ 70000	High
Salary ≥ 60000 (but below 70000)	Medium
Anything lower	Low

EXAMPLE

Name	Salary	SalaryBand
Sneha	75000	High
Ravi	65000	Medium
Meera	55000	Low

> **Order of the rules matters**
> Check the highest band first. A salary of 75,000 satisfies both "≥ 70000" and "≥ 60000", so the High test must be applied before the Medium test.

Task 5 · Top-Paid Employees

FUNCTION

```
def top_paid_employees(employee_df, n):
```

PARAMETERS

Parameter	Type	Description
employee_df	DataFrame	The cleaned employee data.
n	integer	How many top earners to return.

WHAT YOU NEED TO DO

Return the `n` employees with the highest salaries, sorted from highest to lowest.

EXAMPLE

Input: `top_paid_employees(employee_df, 3)`

Output:

Name	Salary
Sneha	75000
Ravi	65000
Amit	62000

Task 6 · Location Summary

FUNCTION

```
def location_summary(employee_df):
```

WHAT YOU NEED TO DO

Group employees by their office Location and produce one summary row per location. For each location, calculate:

- Employee Count — how many employees work there.
- Average Salary — the mean salary at that location.

EXPECTED OUTPUT

Location	EmployeeCount	AverageSalary
Mumbai	3	61166
Pune	3	55000

> **Tip**
> Same grouping idea as the department report, but grouped by Location instead of Department. The average may not be a round number, which is expected.

Task 7 · Salary Ranking

FUNCTION

```
def salary_ranking(employee_df):
```

WHAT YOU NEED TO DO

Add a new column called `SalaryRank` that ranks employees by salary, where the highest-paid employee gets rank 1, the next highest gets rank 2, and so on.

EXAMPLE

Name	Salary	SalaryRank
Sneha	75000	1
Ravi	65000	2
Amit	62000	3

> **Direction of the rank**
> Rank 1 must go to the HIGHEST salary (descending order), not the lowest. Make sure the ranking is set up so the top earner is ranked first.

Interview Questions

Q1. Why use fillna() for salary?

Answer: Missing salaries distort reports and averages. Filling them (here, with the average salary) keeps those records usable instead of dropping them.

Q2. Why use apply() for salary banding?

Answer: It lets you apply a custom business rule to each row individually — ideal for converting a salary number into a High / Medium / Low label.

Q3. Difference between sort_values() and nlargest()?

Answer: sort_values() sorts the entire dataset; nlargest() directly returns just the top-N rows, which is more efficient when you only need a few.

Q4. Why use groupby()?

Answer: To build department-wise and location-wise summaries by collapsing many rows into one row per group.

Q5. Why use rank()?

Answer: To assign ranking positions (1st, 2nd, 3rd ...) to employees based on salary.

Challenge Extensions

Stretch tasks once the core functions work.

LEVEL 2

```
def department_pivot(employee_df):
```

Generate a Location-vs-Department pivot table showing the employee count for each combination of location and department. (Rows = locations, columns = departments, values = counts.)

LEVEL 3

```
def salary_distribution(employee_df):
```

Return how many employees fall into each salary band — i.e. the count of `High`, `Medium`, and `Low` employees. (Build on the SalaryBand column from Task 4.)

LEVEL 4

```
def department_top_employee(employee_df):
```

Find the highest-paid employee within each department — one top earner per department.